

영어 문장 만들기 — 프로젝트 가이드

프로젝트 목적

초등학교 4학년 학생이 단어 카드를 조합해서 영어 문장을 만들고, AI가 문법·문맥 피드백을 주는 학습 앱.

핵심 교육 목표:

- 단어 카드를 슬롯(주어·동사·목적어·부사절 등)에 배치해 문장 구조를 체험
- 완성된 문장을 AI가 평가 — 문법 정확성보다 **의미의 자연스러움** 우선
- 오답 시 왜 어색한지 친근한 말투로 피드백, 대안 문장 2개 추천

폴더 구조

```
프로젝트 루트/  
├── api/  
│   ├── claude.js           # Anthropic API 서버리스 함수 (현재 사용)  
│   └── openai.js           # OpenAI API 서버리스 함수 (예비)  
├── src/  
│   ├── App.jsx             # 앱 루트 — 전체 상태 관리  
│   ├── main.jsx            # React 진입점  
│   ├── components/  
│   │   ├── TopBar.jsx      # 상단바 — 문장 미리보기 + 데이터셋 선택  
│   │   ├── Board.jsx       # 카드 컬럼 전체 렌더링 + 데이터 로딩  
│   │   ├── CardColumn.jsx  # 슬롯 1개 = 컬럼 1개  
│   │   ├── WordCard.jsx    # 카드 1장 렌더링  
│   │   ├── ControlPanel.jsx # 레이어 탭 + 제출 버튼 + AI 호출  
│   │   └── FeedbackModal.jsx # 채점 결과 모달  
│   ├── styles/  
│   │   ├── App.css         # 전역 CSS 변수 + body/root 기본 설정  
│   │   ├── TopBar.css  
│   │   ├── Board.css  
│   │   ├── CardColumn.css  
│   │   ├── WordCard.css  
│   │   ├── ControlPanel.css  
│   │   └── FeedbackModal.css  
│   ├── data/  
│   │   └── set1/  
│   │       ├── subject.json # 주어 카드  
│   │       ├── verb.json    # 동사 카드  
│   │       ├── object.json  # 목적어 카드  
│   │       ├── modal.json   # 조동사 카드  
│   │       ├── adverb.json  # 부사절 카드  
│   │       └── preposition.json # 전치사구 카드
```

```
| | └─ relative.json      # 관계사절 카드
|   └─ utils/
|       └─ verbConjugate.js  # 동사변형 + 접속사 분리 로직
└─ index.html
```

데이터 구조 (JSON 스키마)

subject.json

```
jsonc
{
  "left": [...],    // 2열 레이아웃 - 왼쪽 열
  "right": [...]   // 2열 레이아웃 - 오른쪽 열
}
// 각 항목
{ "kor": "그는/그가", "eng": "He", "isT3": true }
{ "kor": "헨리가", "eng": "Henry", "isT3": true, "isProper": true }
```

필드

설명

(kor)

한글 레이블 (카드에 표시)

(eng)

영어 텍스트

(isT3)

3인칭 단수 여부 → 동사 s형 결정

(isProper)

고유명사 여부 → 부사절 앞배치 시 소문자 변환 방지

(isProper: true) 항목: (Buzz), (Pete), (Froggy), (Henry) (I)는 별도 하드코딩으로 처리 ((You and I
케이스 때문에 JSON에 넣지 않음)

verb.json

```
jsonc
{ "base": "love", "s": "loves", "ed": "loved" }
```

adverb.json

```
jsonc
{ "kor": "자기 전에", "eng": "before I went to bed", "conj": "before" }
```

`conj` 필드: 카드에서 접속사 부분만 파란색으로 강조 표시

`object.json` / `preposition.json`

```
jsonc
{
  "kor": "학교에서",
  "eng": "at school"
}
```

`relative.json`

```
jsonc
{
  "kor": "그들이 싫어하는",
  "eng": "who they hate",
  "canOmit": true
}
{
  "kor": "그 집에 사는",
  "eng": "who lives there",
  "canOmit": false
}
```

`canOmit: true`: 목적격 관계대명사 → 카드에서 `who/which` 부분 반투명 표시, AI에게 생략 가능 표시 전달

`modal.json`

```
jsonc
{
  "kor": "~할 수 있어",
  "eng": "can"
}
```

파일별 상세 설명

`App.jsx` — 상태 허브

앱 전체 상태 보유. 모든 컴포넌트에 `props`로 전달.

상태	타입	설명
<code>layerIdx</code>	<code>number</code>	현재 활성 레이어 인덱스
<code>slotOrder</code>	<code>string[]</code>	현재 레이어의 슬롯 순서 (드래그로 변경 가능)
<code>tense</code>	<code>'present' 'past'</code>	현재형·과거형 (현재 UI에서 변경 불가, 항상 <code>present</code>)
<code>currentSet</code>	<code>string</code>	현재 데이터셋 (<code>'set1'</code>)
<code>selection</code>	<code>{ [slotKey]: { item, slotKey } }</code>	각 슬롯에서 선택된 카드
<code>modal</code>	<code>object null</code>	피드백 모달 데이터

`LAYERS` 배열: 레이어별 슬롯 구성 선언.

```
js
{
  id: 'layer1',
  label: '기본 문장 + 부사절',
  slots: [
    { key: 'adverb', label: '문장 늘리기 (부사절)', color: 'ext', dataFile: 'adverb' },
    { key: 'subject', label: '명사 (주어)', color: 'subj', dataFile: 'subject' },
    { key: 'verb', label: '동사', color: 'verb', dataFile: 'verb' },
    { key: 'object', label: '명사 (목적어/보어)', color: 'obj', dataFile: 'object' }
  ]
}
```

슬롯 속성:

- optional: false → 미선택 시 제출 불가
- isVerb: true → 동사변형 처리 대상
- color → 슬롯별 색상 키 ((subj), (verb), (obj), (mod), (rel), (ext))

`buildColoredNodes()`: 상단바에 표시할 색상 문장 노드 생성. 부사절 앞배치 시 주어 소문자 처리 포함.

`effectiveTense`: 부사절 카드에 `tense` 필드가 있으면 그걸 우선 적용.

`checkRef` / `refreshRef`: `ControlPanel`의 `handleCheck`, `handleRefresh` 함수를 App에서 참조하기 위한 ref.

TopBar.jsx — 상단바

표시 항목:

- 데이터셋 선택 드롭다운 (`set-select`)
- 조합된 문장 미리보기 (`coloredNodes`) — App에서 생성해서 전달

레이어 탭과 시제 스위치는 **ControlPanel**로 이동됨.

Board.jsx — 데이터 로딩 + 컬럼 배치

모든 JSON을 정적 import → DATA_MAP으로 관리:

```
js
DATA_MAP = { subject, verb, object, modal, adverb, preposition, relative }
```

슬롯 순서대로 `CardColumn` 렌더링. 드래그 앤 드롭으로 걸림 순서 변경 가능 (`onSlotReorder`)로 App에 전달).

마우스 휠로 가로 스크롤 지원.

`isAdverbFront`: 첫 번째 슬롯이 `ext` 컬러(부사절/전치사구)이고 선택된 경우 → 주어 카드를 소문자로 표시.

`CardColumn.jsx` — 슬롯 컬럼

- `slot.color` → `COLOR_MAP`으로 컬럼 CSS 클래스 결정
- JSON 구조가 `{ left, right }` → 2열 그리드
- JSON이 배열 → 1열 (`adverb`, `modal` 등)
- 드래그 이벤트 처리 (`draggable`, `onDragStart`, `onDrop`, `onDragEnd`)

`WordCard.jsx` — 카드 1장 ★

슬롯 종류별 렌더링:

슬롯	한글 표시	영어 표시
<code>verb</code>	✗	<code>verbForm.base</code> + <code>suffix</code> (빨강)
<code>adverb</code>	✓	<code>conj</code> + <code>base</code> (파랑)
<code>preposition</code> , <code>subRel</code> , <code>objRel</code> , <code>modal</code>	✓	<code>item.eng</code> 그대로
<code>relative</code> (<code>canOmit</code>)	✓	who/which 반투명 + <code>(=that)</code> 표시
나머지	✗	<code>item.eng</code> 그대로

부사절 앞배치 시 주어 카드: `toMidSentence()`로 소문자 변환 (고유명사/I 제외).

`ControlPanel.jsx` — 문장 조합 + AI 채점 ★

레이아웃: 하단 고정. 왼쪽에 레이어 탭, 오른쪽에 초기화·문장확인 버튼.

주요 함수:

`buildParts()` — 선택된 슬롯을 순서대로 파트 배열로 변환:

```
js
```

```
[{ text, slotKey, canOmit, isProper }, ...]
```

`buildSentence()` — 파트를 문장으로 조합:

- 부사절 앞배치: `"adverbText, mainClause."`
- 주어 소문자 처리: `isProper`가 없고 `I`가 아닌 경우만 소문자 변환
- 첫 글자 대문자, 마침표 추가

`sentenceForAI` — AI에 전달하는 문장:

- `sentence` 기반으로 `canOmit` 슬롯 텍스트를 `[...]`로 감쌈
- 예: `"Froggy [who they hate] gives a book."`
- AI에게 생략 가능 관계사임을 알려주기 위함

`validate()` — `optional: false` 슬롯 모두 선택됐는지 확인

`callClaude()` — `/api/claude` 호출, JSON 파싱 반환

`handleCheck()` — 채점 실행:

1. `validate` 통과 여부 확인
2. `sentence` 생성
3. `sentenceForAI` 생성
4. 프롬프트 구성 → `callClaude()` 호출
5. 결과를 `onResult`로 App에 전달
6. 캐시: 같은 문장 재제출 시 API 호출 없이 캐시된 결과 표시

`handleRefresh()` — 추천 문장 새로고침:

- 현재 추천 2개를 제외하고 새로운 추천 2개 요청

result 객체 구조:

```
js
```

```
{
  isCorrect,           // boolean
  sentence,            // 완성된 영문 (대소문자 처리 완료)
  wrongWords,          // 틀린 단어/구 배열
  wordSlots,           // buildParts() 결과 - FeedbackModal S/V/O 레이블용
  isAdverbFront,       // 부사절 앞배치 여부 - FeedbackModal 대소문자 처리용
}
```

```

correctWords,      // [] (현재 미사용)
translation,       // 한국어 번역
explanation,         // 피드백 (HTML span 포함 가능)
recommendedSentence, // 추천 문장 1
recommendedSentence2, // 추천 문장 2
rec1Translation,    // 추천 번역 1
rec2Translation,    // 추천 번역 2
}

```

FeedbackModal.jsx — 채점 결과 모달

헤더: 정답(success 초록) / 오답(warn 오렌지)

문장 표시 (wordSlots 기반):

- 첫 번째 파트: 대문자
- isAdverbFront이고 isProper가 아니고 I가 아닌 경우: 소문자
- canOmit 파트: who/which 반투명 표시
- wrongWords에 포함된 파트: .word-err 오렌지 색상

explanation 렌더링:

- \\n으로 줄바꿈 분리
- highlightEn(): 따옴표 안 영단어 → wrongWords면 오렌지, correctWords면 파랑

추천 문장: 방법 1, 방법 2 카드로 표시

푸터:

- 정답: 다음 문장도 도전! →
- 오답: 다시 해볼게! + 추천 문장 ↺

verbConjugate.js — 변형 로직

함수	용도	반환
getVerbForm(verbObj, subject, tense, hasModal)	카드 렌더링용 — 변형 분리	{ base, suffix }
getPlainVerb(verbObj, subject, tense, hasModal)	문장 조합용	string
getAdverbForm(adverbObj)	접속사 분리	{ conj, base }

`splitForm(base, conjugated)`

공통 앞부분 분리 (내부
용)

`{ base, suffix
}`

`hasModal: true`면 동사원형 그대로 반환 (조동사 뒤 원형 규칙).

AI 연동

API Route (`/api/claude.js`)

Vercel 서버리스 함수. `/api/claude` 엔드포인트로 노출됨.

환경변수: ANTHROPIC_API_KEY_make_sentence
모델: claude-sonnet-4-6
temperature: 0.2
max_tokens: 요청에서 전달 (512 또는 256)

`/api/openai.js`도 존재 (OpenAI gpt-4o용). 현재는 미사용.

프롬프트 구조 (`handleCheck`)

역할 선언: 초등 4학년 영어 선생님
입력 문장: sentenceForAI (canOmit 슬롯은 [...] 표시)
번역 규칙: 직역, Fly Guy/Girl 예외 처리
평가 기준: 1차 문법 → 2차 문맥 → 3차 문법요소별 → 4차 동물의인화 허용 → 5차 도덕판단 없음
정답 피드백: 친근한 말투, 2문장 이내
오답 피드백: 간결, 틀린단어 오렌지 span, 추천단어 파란색 span
추천 문장 규칙: 원래 단어 유지, 초등 수준, 관계사 생략형 우선
관계사 규칙: [...] = 목적격 관계사, 생략 가능, 오답 처리 금지
출력: JSON만, 마크다운 금지, span은 explanation에서만

JSON 응답 형식:

```
json

{
  "isCorrect": true,
  "translation": "한국어 번역",
  "explanation": "피드백 (HTML span 가능)",
  "wrongWord": null,
  "rec1": "추천 문장 1",
  "rec1kor": "추천 번역 1",
```



```
"rec2": "추천 문장 2",
"rec2kor": "추천 번역 2"
}
```

프롬프트 (`handleRefresh`)

추천 문장 새로고침 전용. 이미 보여준 추천 2개 제외하고 새로운 대안 2개 요청. `max_tokens: 256`.

컴포넌트 연동 흐름

```
App
├─ state: layerIdx, slotOrder, tense, currentSet, selection, modal
├─┐
├─┤ TopBar      (coloredNodes, currentSet, sets → onChange)
├─┤
├─┤ Board      (layer, slotOrder, tense, selection → onSelect,
onSlotReorder)
├─┤   └─┐ CardColumn (slot, data, tense, selectedItem, selectedSubject,
isAdverbFront, hasModal → onSelect)
├─┤       └─ WordCard (item, slot, tense, selectedSubject, isSelected,
isAdverbFront, hasModal → onSelect)
├─┤           └─ verbConjugate.js (getVerbForm, getAdverbForm)
├─┤
├─┤ └─ ControlPanel (layer, slotOrder, tense, selection, layers, layerIdx →
onReset, onResult, onLayerChange, onRegisterCheck, onRegisterRefresh)
├─┤     └─┐ verbConjugate.js (getPlainVerb)
├─┤         └─ /api/claude → AI 피드백
├─┤
├─┤ └─ FeedbackModal (modal → onClose, onRefresh)
```

CSS 변수 (`App.css`)

색상

변수	값	용도
<code>--bg</code>	<input type="color" value="#f4f2eb"/> #f4f2eb	앱 배경
<code>--card</code>	<input type="color" value="#f4f2eb"/> #f4f2eb	카드/패널 배경
<code>--btn-bg</code>	<input type="color" value="ffffff"/> #ffffff	버튼 배경
<code>--btn-border</code>	<input type="color" value="bebebe"/> #bebebe	버튼 테두리

<code>--text-primary</code>	 #2d2f33	본문 텍스트
<code>--text-secondary</code>	 #7c7f82	보조 텍스트
<code>--divider</code>	 #DEE1E6	구분선
<code>--correct</code>	 #0084ea	정답/추천 색 (파랑)
<code>--correct-bg</code>	 #F1F8FD	정답 배경
<code>--wrong</code>	 #DE3B40	오답/틀린단어 색 (빨강)

슬롯 컬러 (카드 배경)

변수	슬롯
<code>--c-yellow</code>	주어 (subj)
<code>--c-pink</code>	동사 (verb)
<code>--c-blue</code>	목적어 (obj)
<code>--c-green</code>	조동사 (mod)
<code>--c-cream</code>	관계사 (rel)
<code>--c-grey</code>	부사절/전치사구 (ext)

슬롯 컬러2 (상단 문장 미리보기 단어 색)

변수	슬롯
<code>--c-yellow2</code>	주어
<code>--c-pink2</code>	동사
<code>--c-blue2</code>	목적어
<code>--c-green2</code>	조동사
<code>--c-cream2</code>	관계사
<code>--c-grey2</code>	부사절/전치사구

레이어 추가하는 법

`App.jsx`의 `LAYERS` 배열에 추가:

```
js
{
  id: 'layer4',
  label: '새 레이어 이름',
  slots: [
    { key: 'subject', label: '명사 (주어)', color: 'subj', dataFile: 'subject', opt
    { key: 'verb', label: '동사', color: 'verb', dataFile: 'verb', op
    { key: 'object', label: '목적어', color: 'obj', dataFile: 'object', opt
  ]
}
```

`dataFile` 값은 `Board.jsx`의 `DATA_MAP` 키와 일치해야 한다.

새 데이터 파일 추가하는 법

1. `src/data/set1/newfile.json` 생성
 2. `Board.jsx`에서 `import` 후 `DATA_MAP`에 추가
 3. `LAYERS`의 슬롯 `dataFile: 'newfile'`로 참조
-

주요 동작 원칙 / 주의사항

대소문자 처리

- 문장 첫 글자: 항상 대문자
- 부사절 앞배치 시 주어: `isProper: true` 또는 `I`/`I`로 시작하는 경우만 원형 유지, 나머지는 소문자
- 고유명사 (`isProper: true`): Buzz, Pete, Froggy, Henry
- `You and I` 케이스 때문에 `I`는 `subject.json`에 `isProper` 추가 불가, 하드코딩 처리

관계사 `canOmit` 처리

- `relative.json`의 `canOmit: true` = 목적격 관계대명사 (생략 가능)
- 카드 UI: who/which 반투명 표시
- AI 전달: `sentenceForAI`에서 해당 텍스트를 `[...]`로 감쌘
- AI 피드백: 생략 가능함을 안내, 오답 처리 금지

캐시

- 같은 문장을 재제출하면 API 호출 없이 `lastResultRef`의 캐시 결과 즉시 표시

채점 기준

- 문법 정확성 + 문맥 자연스러움 모두 평가
- 동물 의인화 허용
- 도덕적 판단 없음
- 단순히 혼하지 않은 상황은 정답 처리

배포

- Vercel + GitHub 연동 자동 배포
- `/api/` 폴더 안 파일이 자동으로 서버리스 함수로 등록
- 환경변수 추가 후 반드시 Redeploy 필요